# 4. Multimodal Visual Understanding + SLAM Navigation (Voice Version)

## 1. Course Content

1. Learn how the large model integrates voice interaction, multi-point autonomous navigation, physical space semantic mapping, and multimodal visual perception;
2. Master the point calibration of the map semantic mapping file `map_mapping.yaml` and the Dify variable synchronization mechanism;
3. Learn to perform initial pose calibration (2D Pose Estimate) in the RViz2 visualization interface;
4. Issue complex long-duration tasks through voice dialogue: autonomous cross-room patrol, image-aware multimodal memory, returning along the original route, and outputting a detailed report through speech synthesis (TTS);
5. Analyze the core source code of the underlying navigation Action client and the coordinated scheduling of the action service side.

---

## 2. Preparation

### 2.1 Description

> This lesson is applicable to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You need to first enter the Docker container before executing ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Directly run the commands in the vehicle terminal; there is no need to enter Docker.

### 2.2 Start the Dify Local Large Model Environment

Enter the following command in the host terminal to start the Dify service:

```
sh ~/bringup_dify.sh
```

The terminal prints the following information, indicating that the Dify service is connected successfully:



> **Important**
> **Prerequisite**:
> This lesson requires the robot to navigate on an already-built map. Please ensure that you have built and saved a grid map (such as `map.yaml`) according to the navigation chapter tutorial.

---

## 2.3 Configure the Map Point Semantic Mapping (map_mapping.yaml)

In order for the large model to understand the physical coordinates of natural-language locations such as "pantry" and "office", the mapping relationship between location names and absolute map coordinates needs to be established in the configuration file.

### 1. Calibrate the Room Coordinates and Orientation

Connect to the robot desktop via VNC and run the following commands to start the navigation nodes:

- **Start the low-level driver and lidar**:

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

- **Start the visualization node (or start the display on the virtual machine side)**:

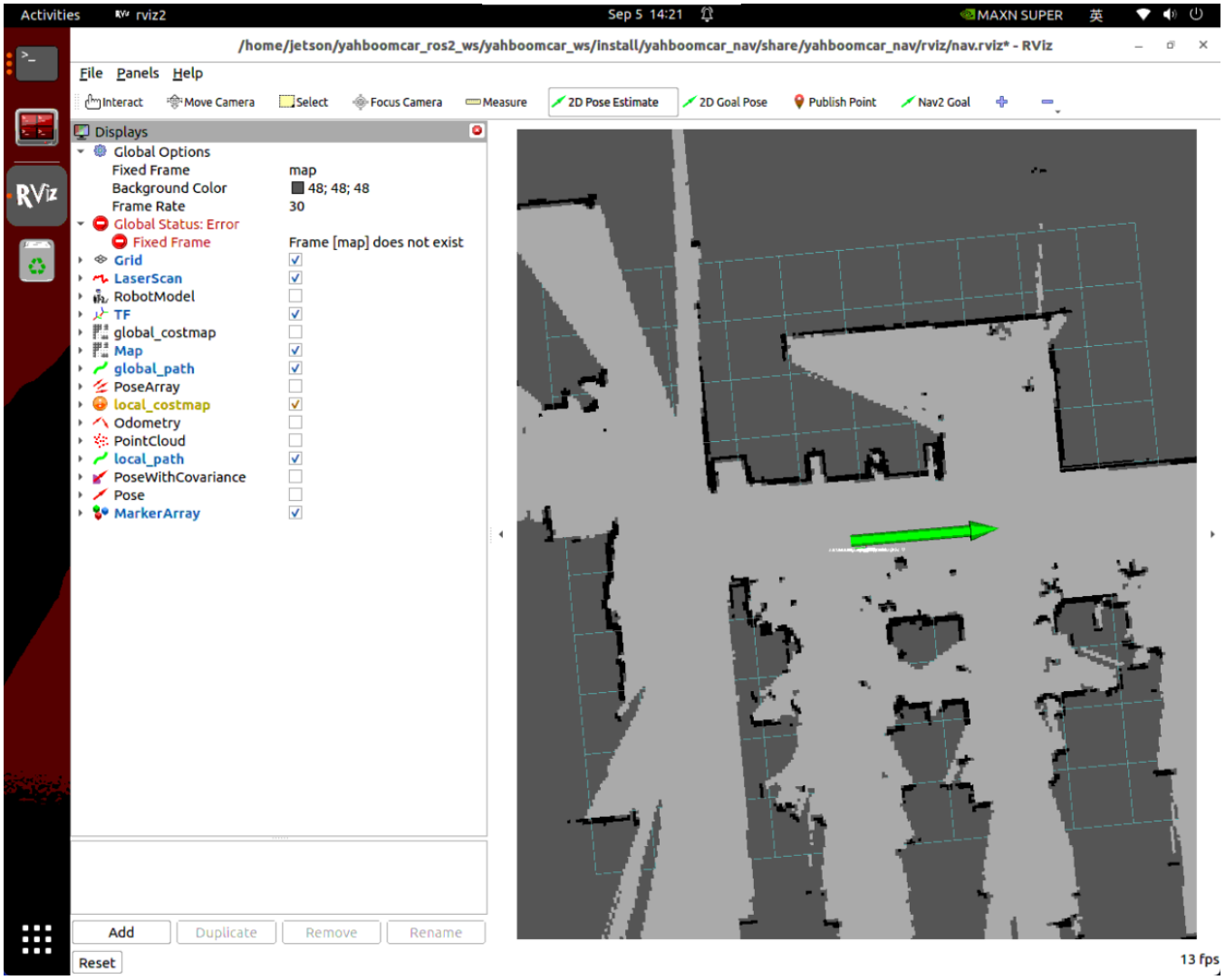
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

- **Start navigation**: **Note: Whichever localization method was used during point calibration must be used for subsequent navigation, because the pose may differ for each localization method**

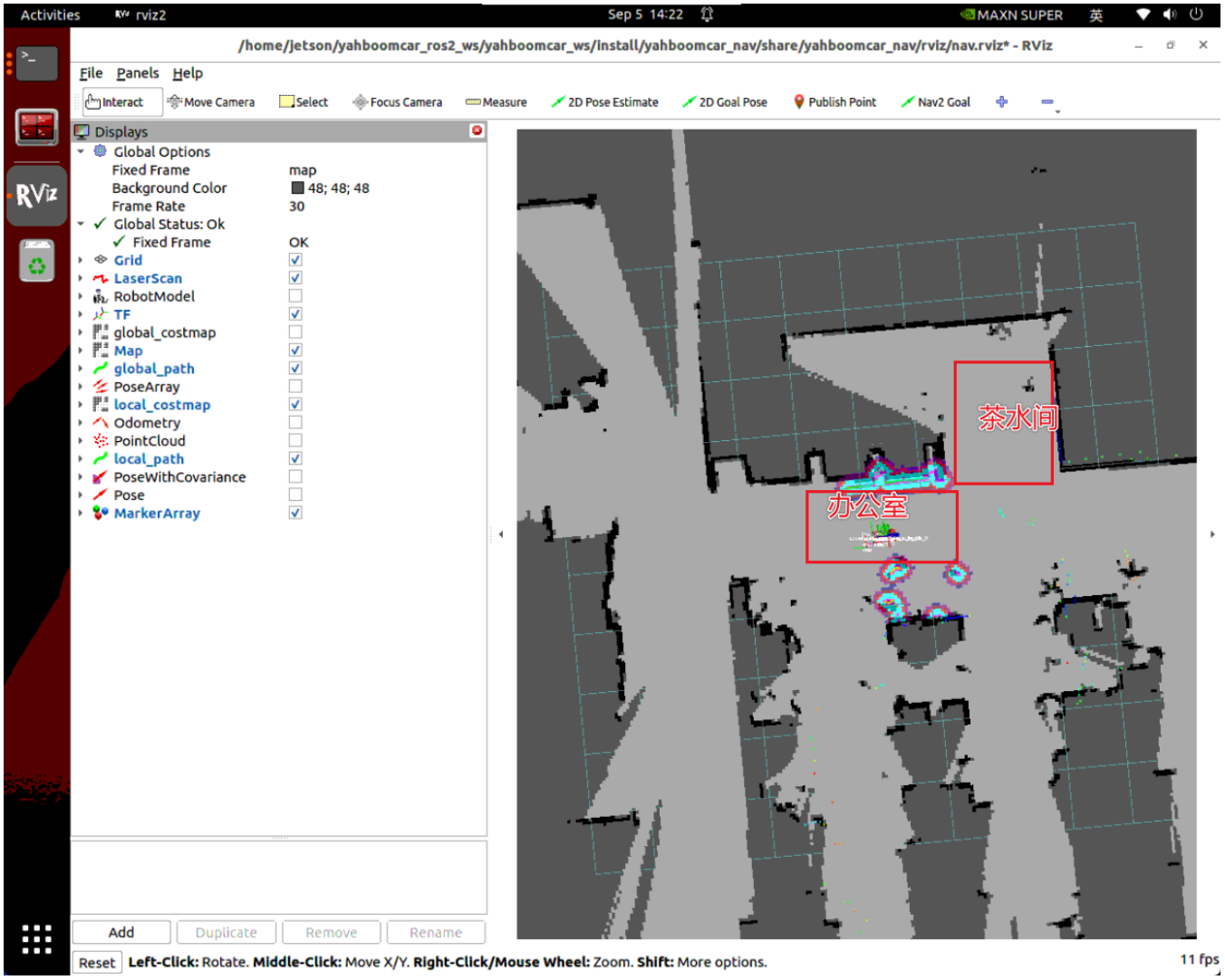
```
ros2 launch yahboomcar_nav navigation_teb_launch.py

# If using the Cartographer relocalization navigation method:
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pbst
ream
ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
```

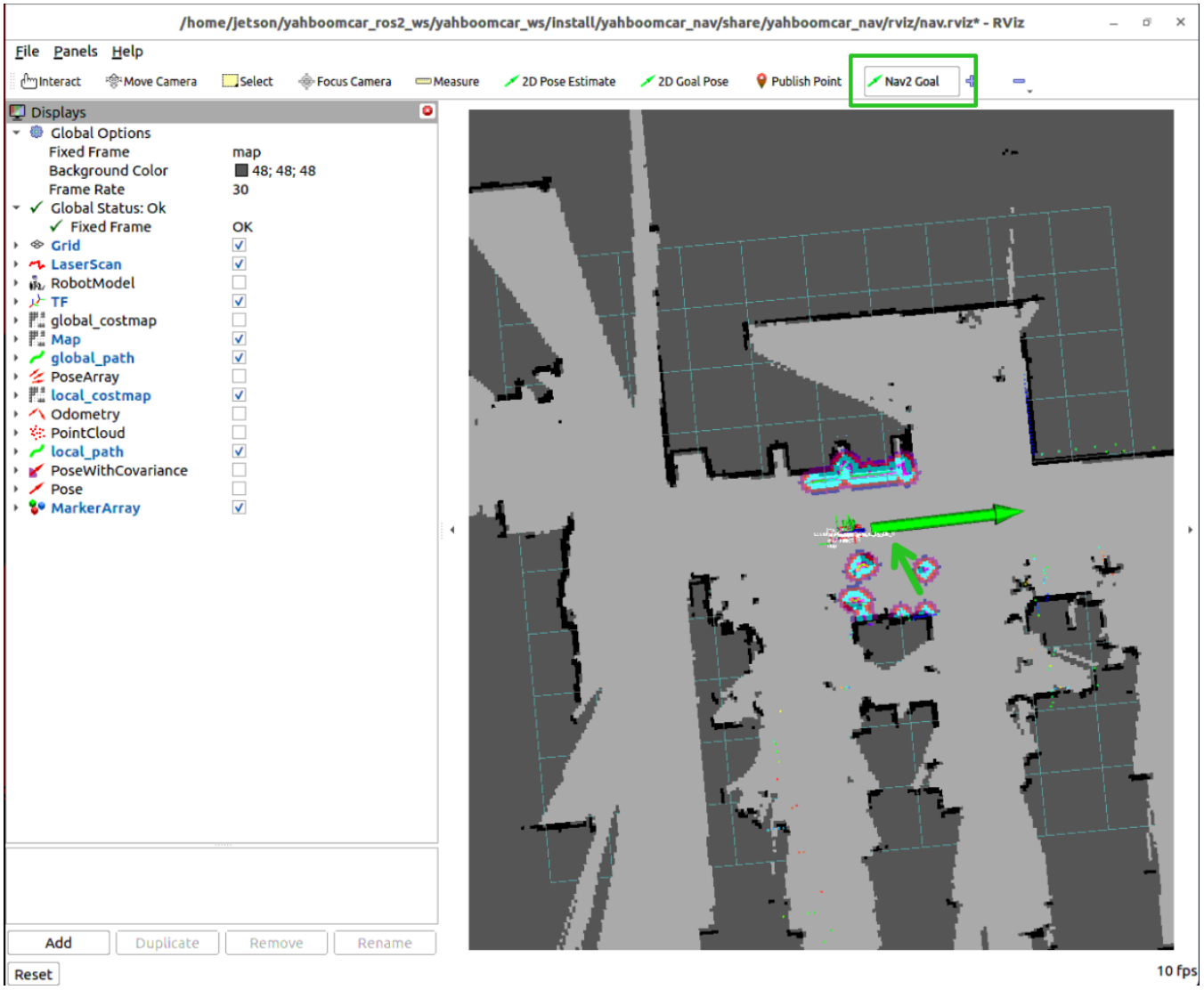
Open the RViz2 navigation interface, click the **2D Pose Estimate** tool in the toolbar at the top, and mark the real physical position and orientation of the car on the map:




Consider the target areas of the car on the map (for example, "office", "pantry"):




In RViz2, use the **2D Goal POSE** tool to click the target location of the room, driving the robot to navigate to that point:




After the robot arrives at the target room, run the following command in the terminal to obtain the precise pose parameters of the car in the `map` coordinate frame:

```
ros2 run tf2_ros tf2_echo map base_footprint
```

The terminal prints the translation `Translation [x, y, z]` and quaternion rotation `Rotation [x, y, z, w]` of the current pose:

```
jetson@yahboom:~/yahboomcar_ros2_ws/software/library_ws$ ros2 run tf2_ros tf2_echo map base_footprint
[INFO] [1788590003.613868389] [tf2_echo]: Waiting for transform map ->  base_footprint: Invalid frame ID "map" passed to canTransform argument target_fram
e does not exist
At time 1788590005.493515420
- Translation: [0.206, -0.598, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.737, 0.676]
- Rotation: in RPY (radian) [0.000, 0.000, -1.656]
- Rotation: in RPY (degree) [0.000, 0.000, -94.883]
- Matrix:
 -0.085  0.996  0.000  0.206
 -0.996 -0.085 -0.000 -0.598
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1788590006.477053138
- Translation: [0.206, -0.598, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.737, 0.676]
- Rotation: in RPY (radian) [0.000, 0.000, -1.656]
- Rotation: in RPY (degree) [0.000, 0.000, -94.883]
- Matrix:
 -0.085  0.996  0.000  0.206
 -0.996 -0.085 -0.000 -0.598
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
```

## 2. Edit the map_mapping.yaml File

Configuration file path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/config/map_mapping.yaml
```

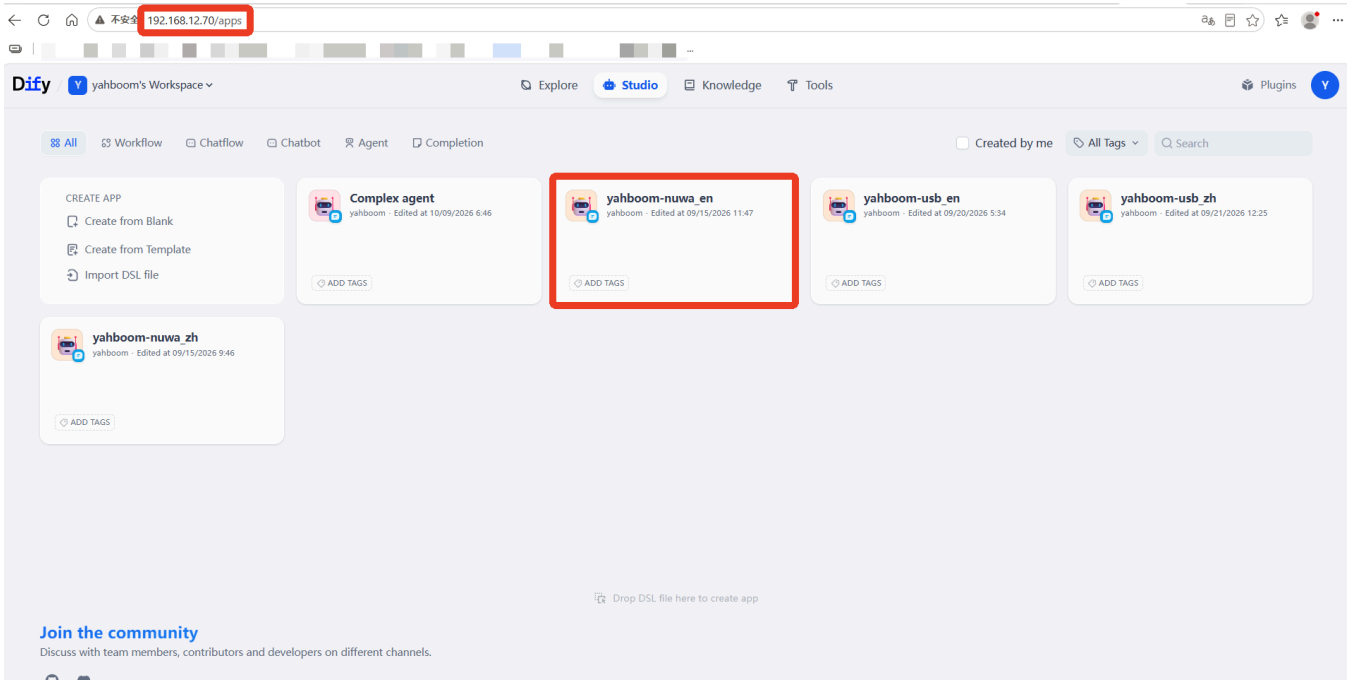
Open the `map_mapping.yaml` map mapping file, and under `common_map_areas` fill in the room names and coordinates obtained just now:

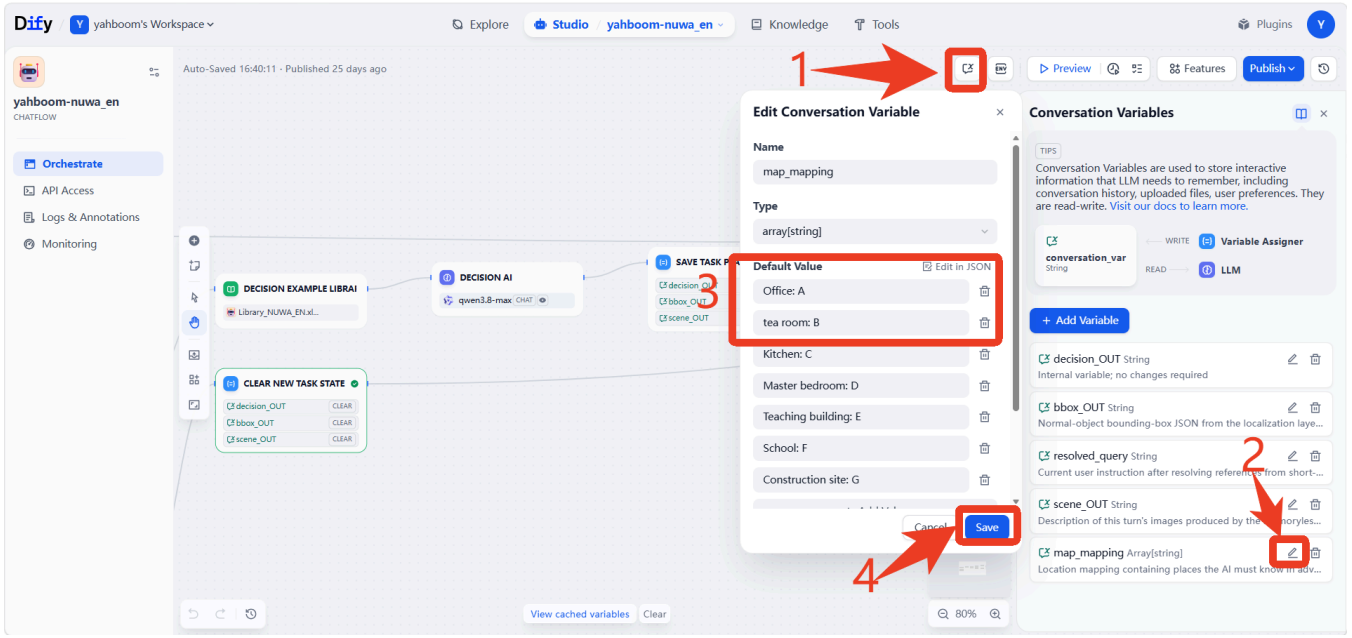
```yaml
common_map_areas: # General navigation
  A:
    name: 'Office'
    position:
      x: 0.206
      y: -0.598
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: -0.737
      w: 0.676
  B:
    name: 'tea room'
    position:
      x: 1.064
      y: -2.432
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: -0.141
      w: 0.990
```

After configuration is complete, recompile and apply the environment variables:

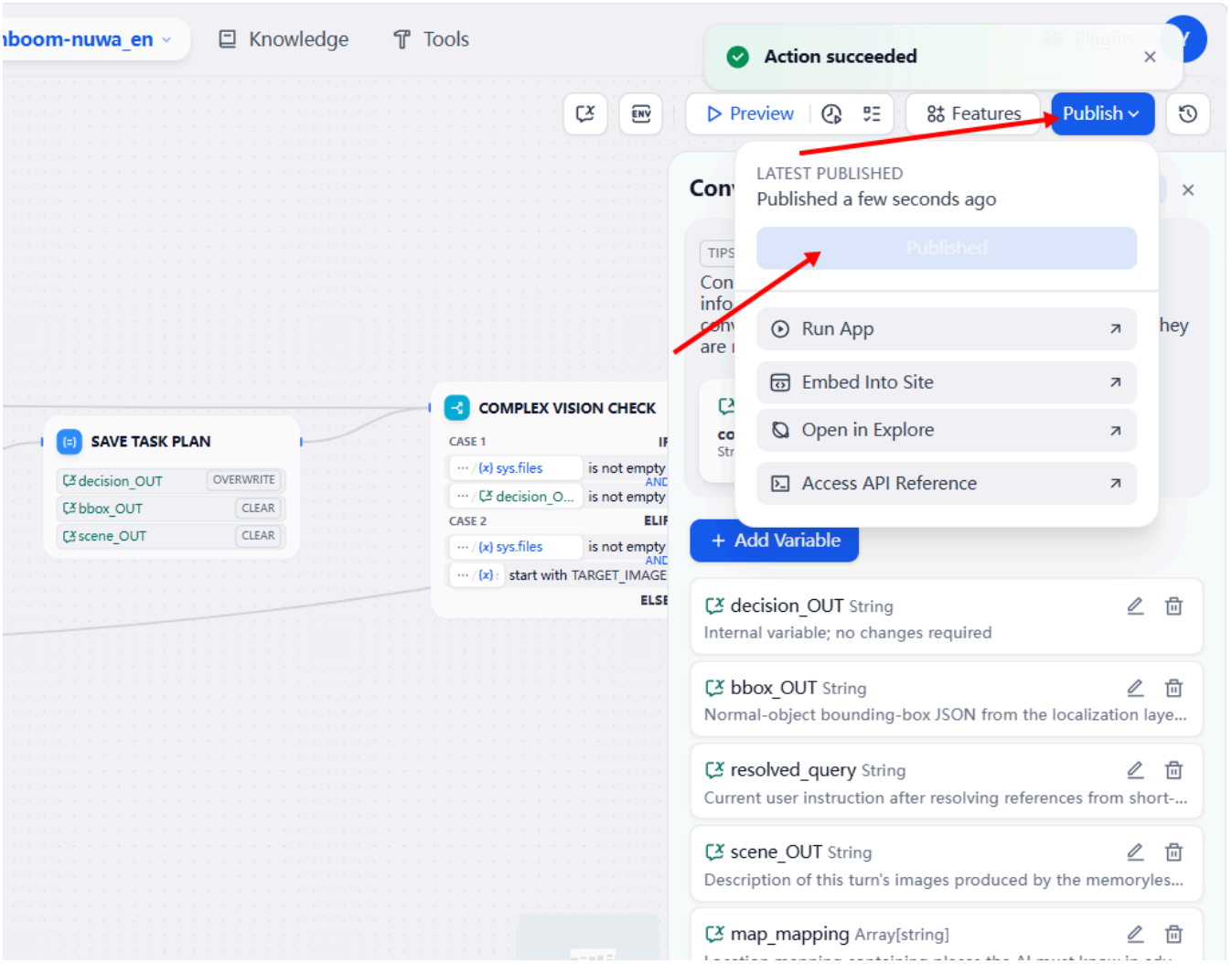
```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/
colcon build --packages-select multi_brains
source install/setup.bash
```

Then, on a LAN computer, open the Dify workbench application (`http://<vehicle IP>`) in a browser, and synchronously update the `map_mapping` variable in the [Conversation Variables] of the corresponding application:




Click [Publish Update]:



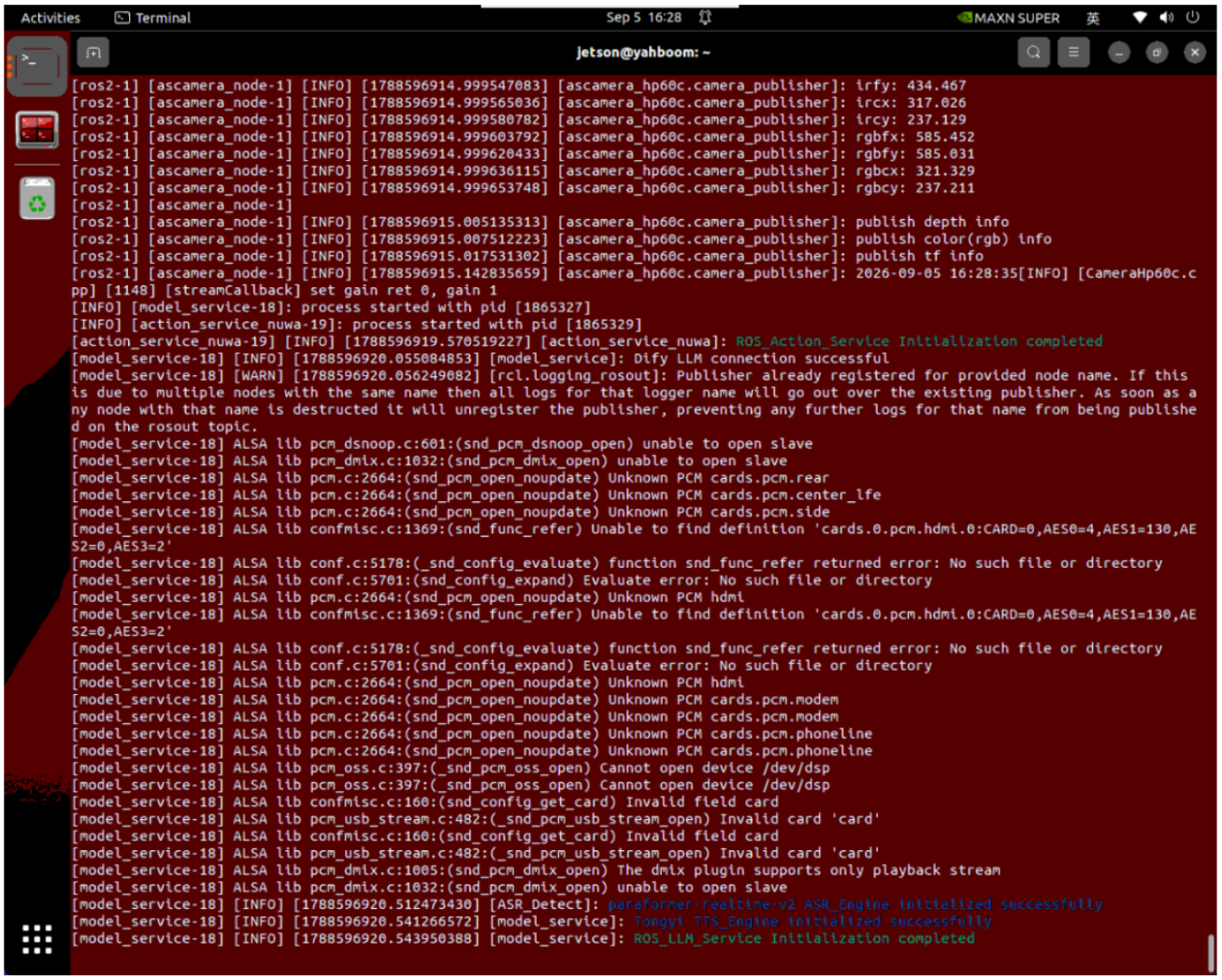

# 3. Running the Cases

## 3.1 Start the Embodied Intelligence Master Control Node (Voice Mode)

Enter the following command in the vehicle terminal to start the large model voice embodied control service:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After the node is ready, the terminal prints the following information:




## 3.2 Start the SLAM Navigation and Visualization System

On the vehicle desktop or through VNC, open another terminal and start the navigation visualization interface:

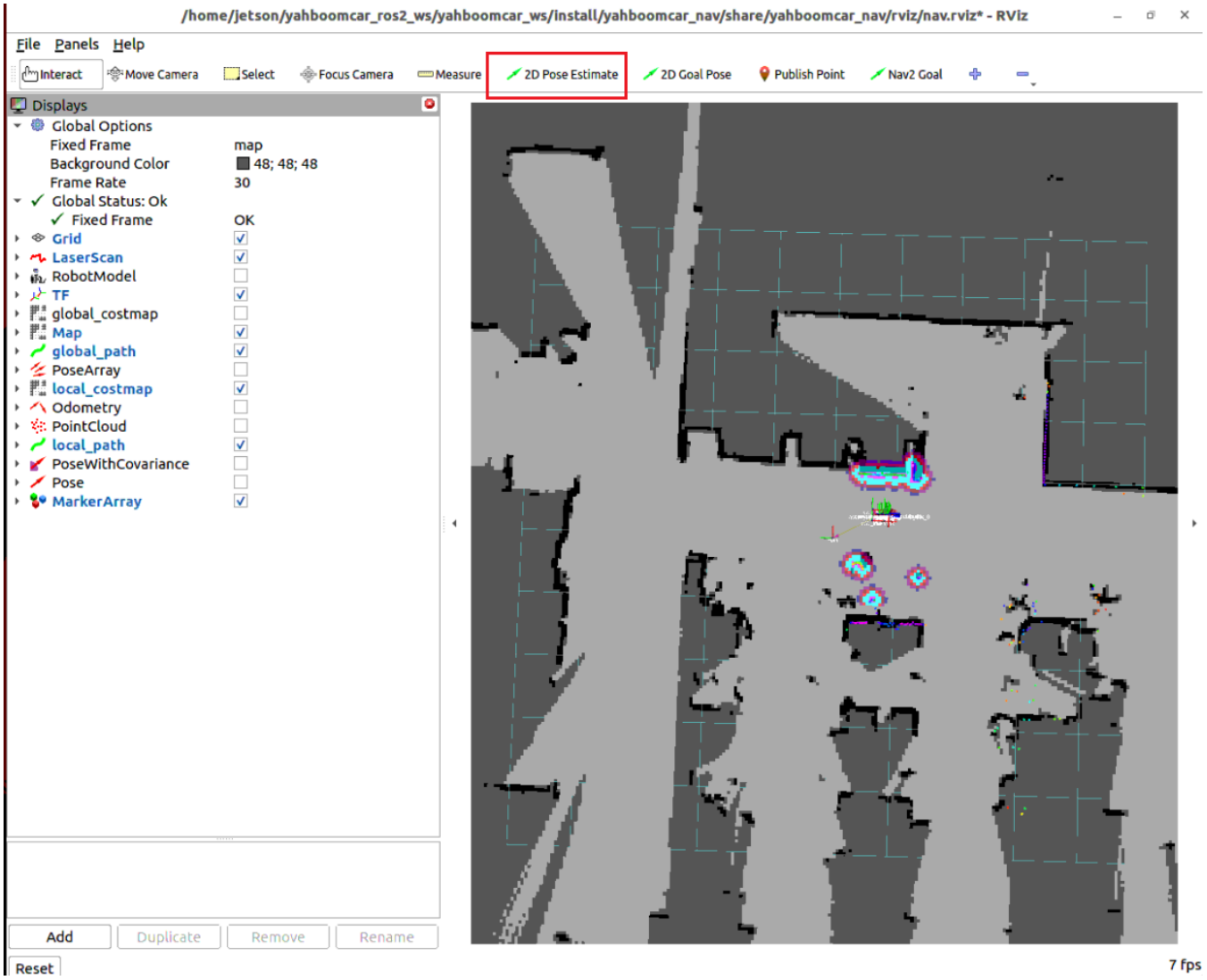
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

Then open another terminal and start the underlying navigation algorithm (the TEB local path planner is recommended): **Note: Whichever localization method was used during point calibration must be used for subsequent navigation, because the pose may differ for each localization method**

```
ros2 launch yahboomcar_nav navigation_teb_launch.py
#Relocalization method; for details, refer to Cartographer relocalization navigation
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.pbstream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py
maps:=$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map.yaml
```

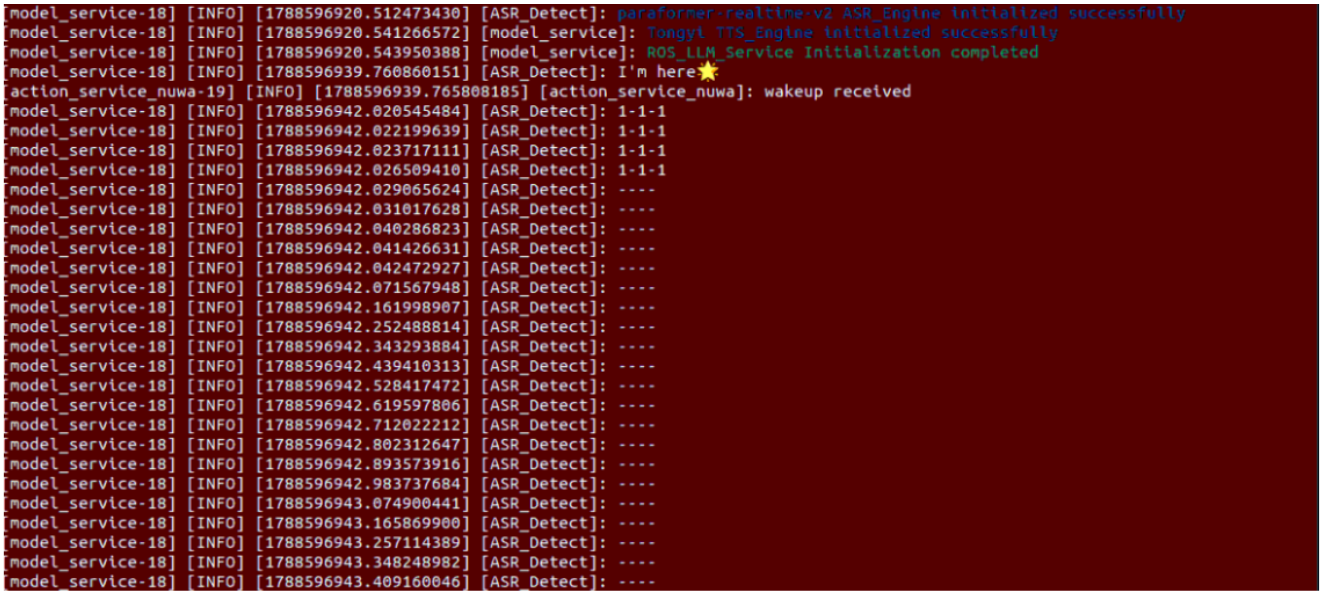
In the opened RViz2 window, use the **2D Pose Estimate** tool to perform initial alignment and localization according to the parking position of the car in the real scene:




## 3.3 Voice Interaction Specification

In voice mode, there is no need to open a text chat node; interact directly with the car's microphone by voice:

1. First clearly call the wake word: **"Hi,yahboom"** (or "yahboom, yahboom");
2. Listen for the car's voice response: **"I'm here, please give me instructions"** or various personalized wake-up voices;
3. After the response ends, you can speak the test instruction to the robot.




---

## 3.4 Classic Test Case (Cross-Room Patrol and Multimodal Long Memory Report)

After waking up yahboom, speak the composite inspection task instruction to the car:

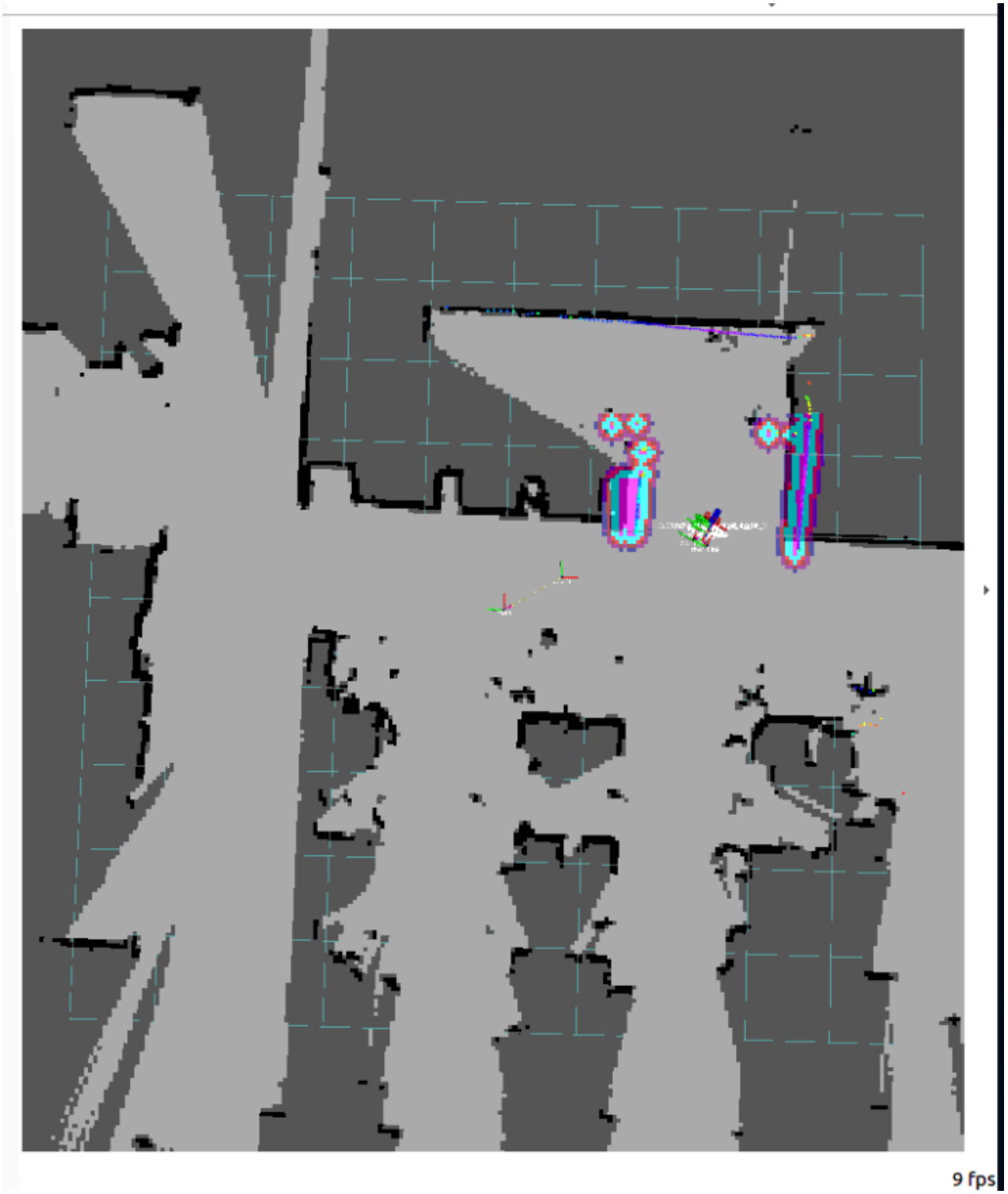
```
Go to the pantry, help me see what items are inside, then return to the office and report
```

The terminal prints the continuous long-chain task plan parsed by the large model:




**Execution Process and Phenomena**:

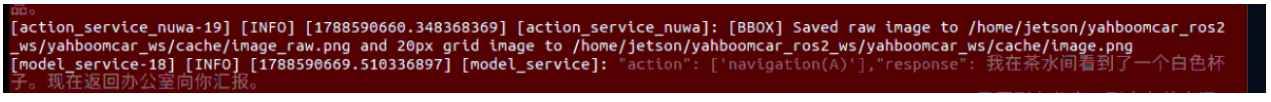
1. **Autonomous navigation to the pantry**: The car chassis automatically plans a global and local obstacle-avoidance path and travels smoothly to the "pantry" point:



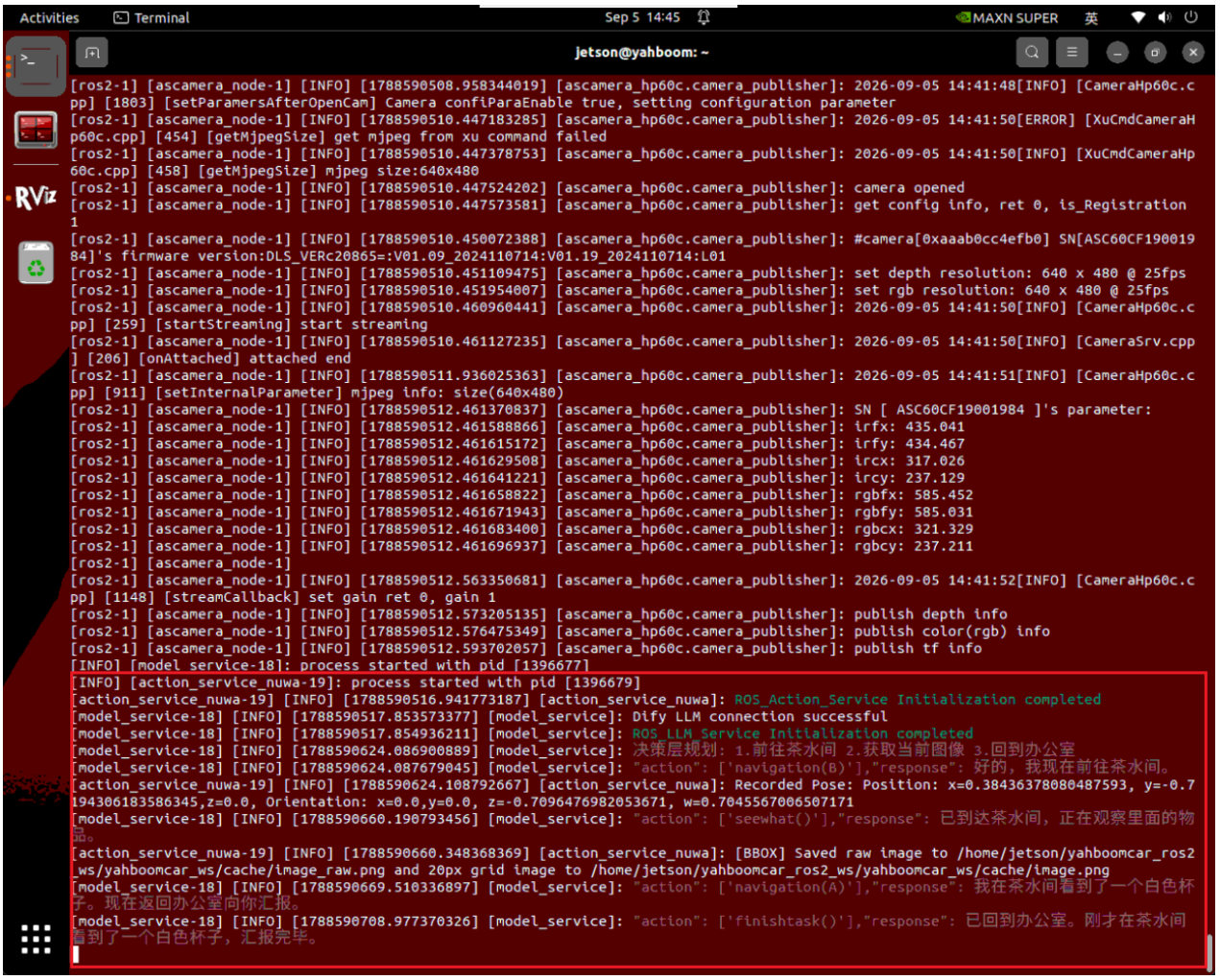

2. **Visual perception and image memory**: After arriving at the pantry, the car brakes and stops, calls the `seewhat()` action to take a photo, and the multimodal large model analyzes the image and recognizes the objects in the scene (such as a tissue box, a drinking cup, colored blocks, etc.). The photo is saved at:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/cache/image_raw.png
```

3. **Autonomous return to the office**: The large model saves the observed object information in the conversation context and automatically plans a path to return to the starting point "office":



4. **Proactive voice summary report**: After arriving at the office, the robot triggers `finish_dialogue()` and reports the patrol results to the user in detail through speech synthesis (TTS):




# 4. Core Source Code Analysis

Source code package path:
`~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains`

The system provides two independent action execution service sides for different hardware configurations (`action_service_nuwa.py` and `action_service_usb.py`), and the navigation interface is completely unified.

## 4.1 Semantic Point Parsing and Navigation Goal Dispatch (action_service_*.py)

In `ActionController`, the `navigation` method encapsulates the Nav2 `NavigateToPose` action client:

```
def navigation(self, point_name):
    # 1. Look up the pose parameters of the corresponding point in the map mapping table
    target_info = self.navpose_dict.get(point_name)
    if not target_info:
        self.get_logger().error(f"未找到目标点位: {point_name}")
        return False

    if self.first_record:
```

```
        # Record the initial position before departure to facilitate returning along the
original route
        self._record_current_pose()
        self.first_record = False

    # 2. Construct the Nav2 target pose message and send it asynchronously
    goal_msg = NavigateToPose.Goal()
    goal_msg.pose = target_info
    send_goal_future = self.navclient.send_goal_async(goal_msg)
```

- **Asynchronous waiting and status closed loop**: The `navigation()` method suspends and waits for the underlying Nav2 navigation to complete. After the chassis comes to a stable stop, it feeds the status back to the large model, driving the large model to continue the subsequent photo-taking `seewhat()` and return tasks.

## 4.2 Visual Memory and Multimodal Long-Context Reasoning

In long-duration inspection tasks, the large model achieves cross-temporal state persistence through the Dify workflow's memory variable (Conversation Context):

- The list of items obtained in the pantry is cached in the conversation history as state context;
- When the robot returns and arrives at the office, the large model synthesizes the historical visual recognition results with the current task goal, automatically generates a summary description, and outputs it to the user through TTS voice.